

9강 - 수업 DNS

도메인(Domain) : 인터넷에서 특정 웹사이트나 서비스에 접근할 수 있도록 하는 텍스트 기반 주소

DNS

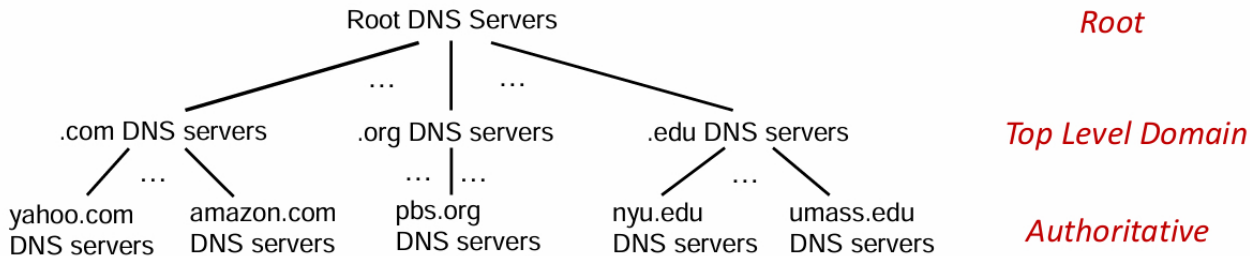
- hostname-to-IP-address translation 호스트이름 -> ip
- canonical names(진짜 이름), alias names(축약된 이름)
- 메일 서버 aliasing
- load distribution(로드 분산)

왜 DNS를 centralize하지 않을까? -> 하나가 작동을 안하면 전체가 작동을 안하기 때문.
트래픽이 너무 몰림, 너무 거리가 멀 수 있음, 유지보수에 용이
확장성이 없어짐.

DNS에 대해 생각해 볼 것

- 거대한 분산 데이터베이스 : 수십억개의 records, 각각은 간단함.
- 하루에 수조개의 쿼리 handle. : 읽기가 쓰기보다 많음.
성능이 중요함.(밀리초 단위) - 거의 모든 인터넷이 DNS로 상호작용하기 때문
- 조직적으로나, 물리적으로 탈중앙화(decentralized)
- "bulletproof" : 안정적(reliability), 보안(security)

DNS distributed, hierarchial database(분산, 계층적 DB)



www.amazon.com의 IP주소를 알고 싶다면?

- ① client queries **root** server to find **.com** DNS server 루트서버에 .com을 물어봄
- ② client queries **.com** DNS server to get **amazon.com** DNS server .com에 amazon.com을 물어봄
- ③ client queries amazon.com DNS server to get IP address for amazon.com에 www.ama.com을 물어봄

DNS Root server

- 이름을 확인할 수 없는 이름 서버에 의한 공식적이고 최후의 연락 수단
- 인터넷 기능에서 매우 중요함. 없으면 인터넷이 동작하지 않음
- DNSSEC** - 인증, 메시지 무결성 등 보안 제공
- ICANN** - root DNS 도메인을 관리

root DNS는 전세계적으로 **13개**가 논리적으로 존재. 물리적으로는 많이 존재. 미국에 200개정도.

TLD Domain과 Authoritative servers

- Top-Level-Domain 서버
- .com, .org, .net, .edu, .aero 등등 + 모든 국가의 top-level 국가 도메인. cn. uk. .fr. ca .jp

네트워크 솔루션: .com .net TLD에 등록이 되어 있음

교육 : .edu

Authoritative(조직) DNS 서버들

조직의 고유한 DNS 서버들. 조직의 host에 대한 ip 매핑에 대한 권한있는 호스트 이름 제공
조직의 서비스 제공자로부터 유지관리됨

호스트가 DNS쿼리를 만들면, local DNS서버로 보내짐.

local DNS 서버는 local cache에서 찾아봄. DNS 계층으로 요청을 보내서 찾음(proxy처럼 작동)
각각의 ISP는 local DNS 서버 이름을 가지고 있음. (ISP가 DNS를 운영하고 있음)

local DNS 서버는 계층에 엄격하게 속하지 않음...?

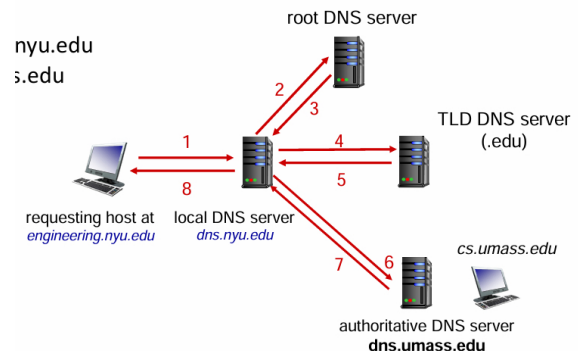
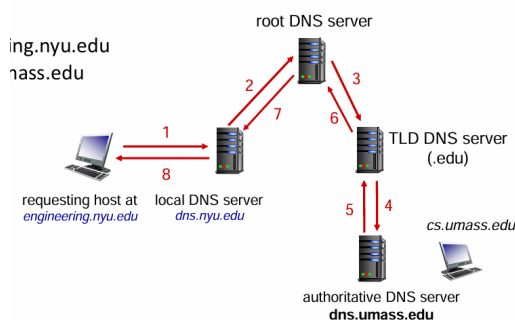
DNS 이름 솔루션 : recursive query 재귀적 쿼리

상황 : engineering.nyu.edu의 호스트가 cs.umass.edu에 접속하고 싶음

Recursive query : 나는 이름을 모르지만, 너를 위해서 해줄게.
연결된 이름 서버에 부담(burden)을 줄 수 있음
계층의 상위 레벨(root DNS 서버)에 부담을 줄 수 있음. heavy load at root server

DNS 이름 솔루션 : iterated query 반복적 쿼리

Iterated query : 나는 이름을 잘 모르지만, 이 서버에 물어봐
local DNS 서버에 물어보면 local DNS서버가 대신해서 해 줌.
light load at root server. 거의 root server에 방문하지 않음. 캐시 (Cache) 때문.



DNS 정보 캐싱(Caching)

일단 이름 서버가 매핑을 학습하면, 캐싱을 해서 기억하고 있다가, 나중에 다시 질의가 오면 즉시 반환한다.
캐싱이 반응 속도를 높인다.

일정 시간(TTL; Time to live, 유효기간) 이후에는 캐시가 Timeout함(사라짐).

TLD 서버는 일반적으로 로컬 이름 서버에 캐시됨.

host가 IP 주소를 변경하는 경우, 모든 TTL이 만료되지 전까지 인터넷 전체에는 알려지지 않을 수 있음.

DNS Records (레코드)

DNS : 리소스 레코드(RR)을 저장하는 분산 데이터베이스

RR(Resource Records) : 도메인의 이름을 저장해 놓은 기록들

RR 형식 : (이름, 값(주소), 타입, ttl(초))

type=A : 이름은 hostname, 값은 IP 주소 e.g. (google.com , 1.1.1.1, A, 300)

type=NS : 이름은 domain(e.g. foo.com) 값은 Authoritative name server의 호스트이름

//도메인을 넣으면 캐리?네임 서버를 알려줌

type=CNAME : 이름은 alias name(canonical의 별칭) 값은 canonical name

//www.ibm.com의 canonical name은 servereast.backup2.ibm.com이다.

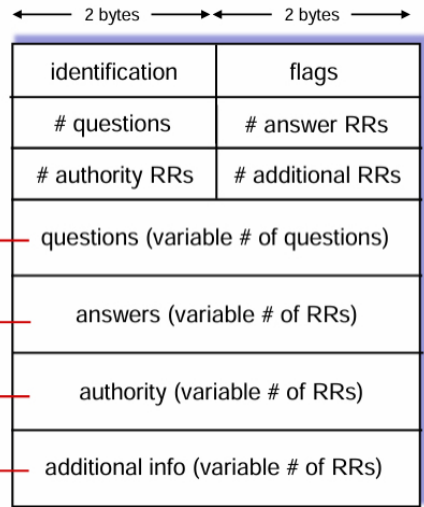
type=MX : 값은 SMTP 메일서버의 domain 이름.

DNS Protocol message(프로토콜 메시지)

DNS 질의(query)와 응답(reply)메시지. 프로토콜은 질의, 응답 다 같은 형식임.

메시지 헤더

- identification(식별자) : 16비트의
- flags(플래그) : 쿼리인지? 응답인지?
recursion을 허용하는지?
authoritative한지(책임이 있는지)?



name, type fields for a query

RRs in response to query

records for authoritative servers

additional “ helpful” info that may be used

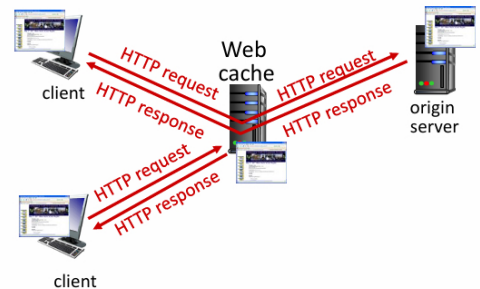
DNS Security

DDoS 공격 : root 서버에 트래픽 왕창 보냄(bombard). 이는 현재는 거의 성공하지 못함. b.o. 트래픽 필터링
TLD 서버에 트래픽 bombard. 잠재적으로 위험할 수 있음.

Spoofing 공격 : DNS 쿼리를 인터셉트해서 bogus(가짜의, 엉처리)응답 함.

DNS 캐시를 poisoning함.

DNSSEC 인증 보안서비스 - RFC 4033



10강 - 수업 Video Streaming and P2P

Web cache 웹 캐시

목표 : 원본 서버 없이 클라이언트의 요청을 만족시키자!

유저가 web cache를 가리키도록 하고, 브라우저가 HTTP요청을 캐시에 보내도록 함.

만약 object가 캐시에 있으면 : 캐시가 클라이언트에게 오브젝트를 리턴

아니면 : 캐시가 원본 서버에 요청을 보내고 받은 후 다시 클라이언트에게 전달.

a.k.a. Proxy(대리인)

웹 캐시는 클라이언트와 서버의 역할을 동시에 한다.

서버는 응답을 보낼 때 캐시 헤더에 캐시가능한 오브젝트인지 아닌지 정보를 담는다

Cache-Control: max-age=<seconds> 얼마 동안 유효함
Cache-Control: no-cache 캐시할 수 없는 정보임.

★자주 바뀌지 않는 내용은 캐시를 해도 되고, 자주 바뀌는 내용은 캐시를 하면 안 됨.

캐시를 하는 이유?

- 클라이언트의 요청에 대한 응답시간을 줄일 수 있음 - 캐시가 클라이언트와 가깝게 있기 때문.
- 기관의 네트워크 트래픽을 줄일 수 있음. -> 네트워크 효율 올라감.
- 인터넷은 캐시로 밀집해짐. poor한 콘텐츠 제공자가 콘텐츠를 더 효율적으로 전달할 수 있다.

Video Streaming and CDNs : 콘텐츠

스트리밍 동영상 트래픽 : 전체 인터넷 bandwidth에서 많은 비중 차지.

넷플릭스, 유튜브, 아마존 프라임이 ISP 트래픽의 80%

challenge : 규모 - 어떻게 수십억명의 사람에게 보내지?

heterogeneity(이형성) : 각각의 유저가 각각의 capability(유무선, 속도 등)이 다름.

solution : 분산형, application-level 인프라 등...

Multimedia : Video

Bit rate : 프로세싱과 전달에 요구되는 데이터 스피드

MPEG1 (CD-ROM) : 1.5Mbps

MPEG2 (DVD) : 3-6Mbps

MPEG4 (주로 인터넷) : 64Kbps-12Mbps

높은 비트레이트(고화질) --- 낮은 비트레이트(저화질)

Streaming stored video

간단한 개요 : 비디오 서버 ---인터넷--> 클라이언트

주요 챌린지 :

server to client의 bandwidth가 계속 바뀜. 네트워크의 congestion level도 계속 바뀜

(집, 액세스네트워크, 네트워크 코어, 비디오 서버에서 다 다름)

congestion(혼잡) 때문에 패킷 로스, 딜레이가 실행을 지연시키고, 결과적으로 동영상 퀄리티가 나빠진다.

Streaming(스트리밍) : at this time, client playing out early part of video, while server still sending later part of video

Streaming stored video : challenges

continuous playout constraint : 플레이 시간과 녹화 당시의 시간(원래 시간)이 같아야 한다...?

네트워크 딜레이가 가변적(jitter)이기 때문에, client-side에 buffer가 필요하다!

*jitter : 네트워크에서 패킷 전송 지연의 변동성

클라이언트와 상호작용 : 일시정지, 빠르게 감기, 되감기, 점프하기 등...

패킷이 lost될 수도 있고, 재전송될 수도 있다.

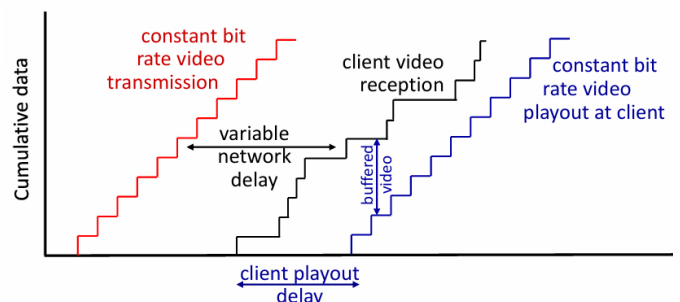
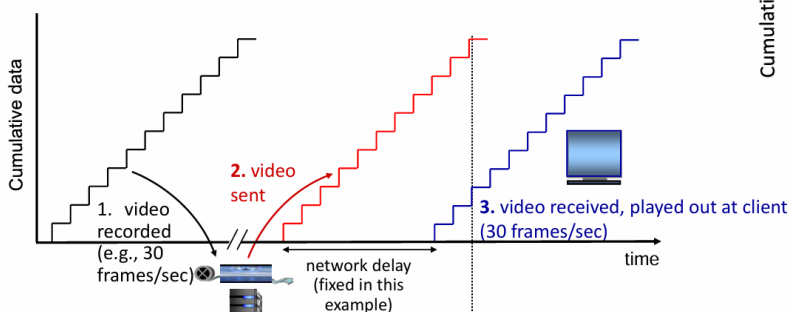
Streaming stored video: playout buffering

buffered video : 컴퓨터 내에 저장되어 있는 동영상

이게 커야 좋음! 작거나 파란색이 더 올라가면 영상이 끊어짐.

client-side buffering과 playout delay : 네트워크에서 추가된 딜레이, 딜레이 jitter를 보완

Streaming stored video (ideal case)



Streaming multimedia: DASH Dynamic, Adaptive, Streaming over HTTP

Server

여러개의 chunks들로 비디오를 나눈다

각각의 chunks들은 다른 레이트(속도)로 각자 네트워크 환경에 맞게 인코딩된다.

서로 다른 속도로 저장된 서로 다른 파일들

다양한 CDN노드에 저장됨

manifest file : 다양한 chunks들에 URL을 제공함. chunks가 어디로 가야할지 길을 제시함.
어느 조각이 어느 위치에 있는지 알려주는 역할

Client

서버-클라이언트 bandwidth를 주기적으로 추정

manifest를 consult. 한번에 한 덩어리씩 요청

현재 bandwidth(네트워크 속도)에 따라 최대 가능한 코딩 속도를 선택

다른 시점에 다른 서버에서, 다른 코딩 속도를 선택할 수 있다 .

클라이언트의 'intelligence'

언제 : 언제 chunk를 요청할 것인가? 버퍼가 starvation(비었을 때) 또는 넘칠 때

어떤 속도(encoding rate)로? : 네트워크 상황에 따라 bitrate(되도록 고화질)를 선택.

어디서 : chunk를 어디서 요청할 것인가? 가깝거나, 가능한 bandwidth가 높은 URL서버에 요청

Streaming video = encoding + DASH + playout buffering

Content distribution networks (CDNs)

: 지리적으로 분산된 여러 사이트에 복사본을 두고 여러 복사본을 저장/제공하는 것

enter deep : 유저와 가까이 있는 것. CDN 서버를 여러 액세스 네트워크 깊숙이 넣음.

Akamai는 120개국에 24만개의 서버를 배치해 놓음.

bring home : 수십개의 작은 클러스터만 두고 있음.

limelight에서 사용

12-1강 - 수업

Video Streaming and P2P

넷플릭스는 어떻게 동작할까?

동영상의 복사본을 Openconnect CDN 노드에 저장함.

사용자가 콘텐츠를 요청하면, 서비스 공급자가 manifest를 리턴한다.

manifest를 이용해서 클라이언트가 가장 빠른 속도를 지원하는 rate로 콘텐츠를 받아온다.

네트워크 경로 congested가 생기면 다른 rate나 copy를 선택할 수 있다.

Content distribution networks (CDNs)

OTT : over the top

OTT(원하는 내용을 인터넷으로 다운로드) vs 케이블 TV(유선으로 모든 가정이 똑같은 신호를 받음)

Internet host-host communication as a service : 대용량 동영상 전송 서비스

OTT의 challenges

edge로부터의 congested Internet에 대응하는 것

어떤 CDN 노드에 어떤 콘텐츠를 놓을까?

어떤 CDN 노드에 있는 콘텐츠를, 어떤 rate로 가져올 것인가?

Client-server paradigm

서버

- always-on host
- 영구적인 IP 주소
- 주로 대규모로 데이터 센터에 있음

클라이언트

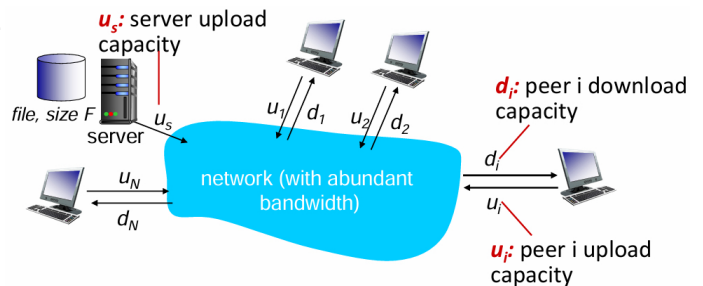
- 서버와 연결하고 통신한다
- 간헐적으로 연결함
- 동적인 IP를 가질 수 있음
- 서로(다른 클라이언트와) 직접적으로 연결하지 않음
- 예 : HTTP, IMAP, FTP

Peer-to-peer (P2P) architecture

- always-on host가 없음
- peer가 다른 peer에서 서비스(파일, 데이터)를 요청하고, 다른 피어로부터 서비스를 제공받음
- self scalability(자가확장성)** : 새로운 peer가 생기면 service capacity가 늘어남. 새로운 서비스 제공 하거나 받음.
- peer들은 간헐적으로 연결되고, IP 주소가 변한다. - 관리하기 복잡함(IP가 계속 바뀌기 때문)
- 예시 : P2P 파일 공유(BitTorrent), 스트리밍(KanKan), VoIP(Skype)

파일 분배 시간: Client-server vs P2P

크기가 F인 파일을 한 서버에서 N개의 peer에게 분배하는데 시간이 얼마나 걸릴까?
peer의 업로드/다운로드 capacity는 한정된 자원이다.

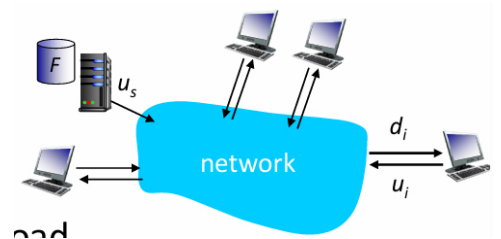


서버 업로드

- N개의 파일 복사본을 반드시 **순차적**으로 보내야 함.
- * 파일 1개를 업로드하는데 걸리는 시간 : F/u_s
- * 파일 N개를 업로드하는데 걸리는 시간 : $N F/u_s$

클라이언트 다운로드

- 각각의 클라이언트는 파일을 다운받아야 함.
- $d_{min} = \min(d_i)$ = 클라이언트 다운로드 속도 중 **최솟값**
- 제일 느린 클라이언트에서 걸리는 시간 : F/d_{min}



N은 클라이언트의 수가 늘어나면 선형적으로 늘어난다!

파일 분배 시간 : Client-server vs P2P

서버 업로드

- 파일을 1번만 올리면 된다!
- * 파일 1개를 업로드하는데 걸리는 시간 : F/u_s

time to distribute F to N clients using client-server approach $D_{C-S} \geq \max\{NF/u_s, F/d_{min}\}$

increases linearly in N

피어

각각의 피어(클라이언트는) 파일을 다운받아야 함.

가장 느린 클라이언트에서 걸리는 시간 : $F/d_{\min}$

모든 피어

모든 피어를 다운로드하는 시간 :

크기(F인 파일을 N개) / 속도(u_s 서버업로드 속도 + $\sum u_i$ (각 peer의 다운속도의 합))

time to distribute F
to N clients using
P2P approach

$$D_{P2P} \geq \max\{F/u_s, F/d_{\min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...

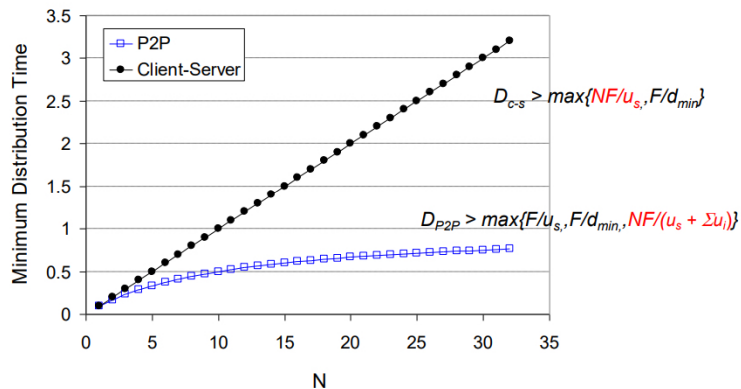
... but so does this, as each peer brings service capacity

파일 크기는 선형적으로 늘지만,

피어 개수가 늘면 업로드 속도가 커져 전체적인 service capacity가 증가한다.

Client-server vs. P2P: example

client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{\min} \geq u_s$



12-2강 - 수업

Video Streaming and P2P

Transport 서비스와 프로토콜

Transport layer는 다른 호스트 간 Application process 사이에서 논리적인 통신을 제공함.

Transport 프로토콜은 end system에서 작동함.

Sender : 메시지를 segment 단위로 나누어 network layer로 전달

Receiver : segment들을 재조합해서 메시지를 만들고, application layer로 전달

Transport 프로토콜은 인터넷 application을 가능하게 해 줌

TCP, UDP

Segment, Packet, Frame

Header 헤더 : 데이터의 앞에 붙어있는 정보 (주로 metadata)

Metadata 메타데이터 : 데이터를 설명하는 데이터 (카메라 종류, 포커스 값, GPS 정보 등등...)

TCP 헤더 : 세그먼트 순서 정보(e.g. 5개 세그먼트 중 2번째)

IP 헤더 : 송신지 IP, 목적지 IP

Segment 세그먼트 e.g. TCP Segment

Transport Layer에서 다루는 내용

데이터를 네트워크를 통한 실질적인 전송을 위하여 적절한 크기로 분할한 조각

Packet 패킷 e.g. IP Packet

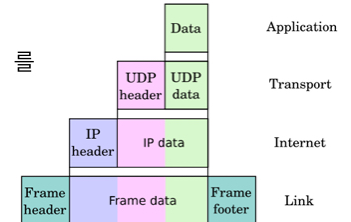
Network Layer에서 다루는 내용

전송을 위해 분할된 데이터 조각(Segment)에 목적지까지의 전달을 위하여 Source IP와 Destination IP가 포함된 IP Header가 붙은 형태의 메시지

Frame 프레임 e.g. 데이터 프레임

Link Layer에서 다루는 내용

최종적으로 데이터를 전송하기 전에 패킷에 MAC 주소를 포함한 헤더와 CRC(Checksome 오류검출)을 위한 Trailer(Footer)가 붙은 메시지



Household analogy

앤의 집에 12명의 아이들이 빌의 집의 8명의 아이들에게 편지를 보낸다.

host = 집 processes = 아이 app message = 편지봉투 안의 편지

transport protocol = 집에 있는 아이들에게 demux하는 앤과 빌

network-layer protocol = 우편 서비스

Transport layer : 프로세스 간 통신 ; relies on, enhances, network layer service

Network layer : 호스트 간 통신

두 가지 Transprot 프로토콜

TCP : Transmission Control Protocol

reliable 믿을 수 있고, in-order delivery 순서대로 전달

congestion control 혼잡 제어

flow control 흐름 제어

connection setup 연결 설정

UDP : User Datagram Protocol

unreliable 믿을 수 없고, unordered delivery 순서없이 전달

no-frills(겉치레가 없는) extension of "best-effort" IP : 보장된 품질 없이 최선을 다해 전송

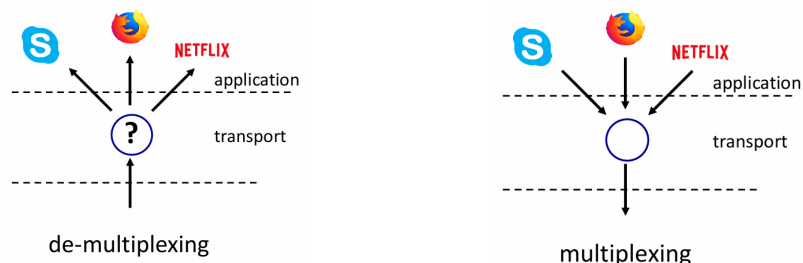
TCP, UDP 둘 다 delay guarantees 지연 보장 X, bandwidth guarantees 속도 보장 X

Multiplexing / Demultiplexing

multiplexing as a sender : 여러 소켓의 데이터 처리, transport header를 추가(나중에 Demultiplexing)

demultiplexing as a receiver : header 정보를 사용하여 수신 segments 올바른 소켓으로 보냄.

헤더에 있는 포트 번호를 보고 어느 프로그램(프로세스)로 보낼 것인가 판단 => De-multiplexing



Demultiplexing의 작동 원리

host가 IP datagram(IP 패킷)을 받는다.

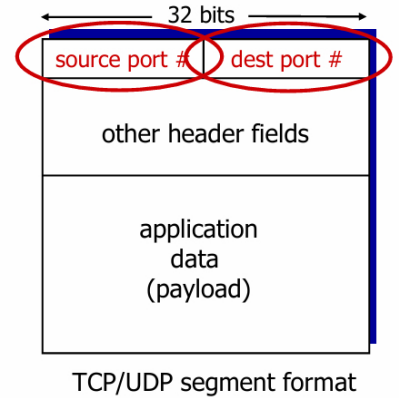
각각의 datagram은 source IP 주소와 destination IP 주소를 가지고 있음

각각의 datagram은 transport-layer의 segment를 가지고 있다.

각각의 segment는 source port와 destination port를 가지고 있음

host는 IP 주소와 포트 번호를 이용해서 segment를 적절한 소켓(앱)으로 보낸다.

IP packet == IP datagram == IP헤더+TCP Segment



UDP: User Datagram Protocol

"no frills", "bare bones"한 인터넷 프로토콜

"best effort" 최선을 다해서 보내는 서비스

UDP segment는 lost 되거나 delivered out-of-order to app 순서가 안 맞을 수 있음.

connectionless :

no handshaking between UDP sender, receiver 핸드셰이킹을 하지 않음. 연결하지 않음.

각각의 UDP segment는 독립적으로 관리된다.

UDP의 특징

연결이 없음. 연결을 하면 RTT delay가 추가됨.

간단함 : sender와 reciever에 연결 상태가 없음

헤더 사이즈가 작음

congestion control이 없음.

원하는 만큼 빠르게 보낼 수 있음. 네트워크 상황에 관여하지 않고.

congestion 상황에서도 작동할 수 있음.

UDP segment header

UDP의 사용

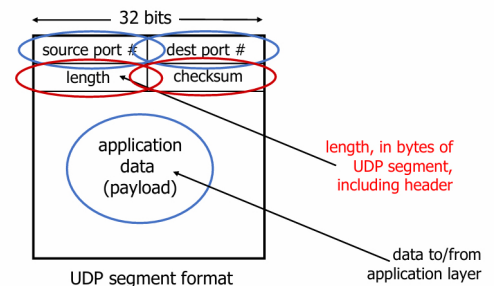
멀티미디어 앱의 스트리밍(loss 허용, rate에 민감)

DNS SNMP HTTP/3

만약 UDP를 통한 안정적인 전송이 필요하다면?

Application layer에서 신뢰성을 추가

Application에서 congestion control을 추가



RFC768에 기록되어 있음.

네트워크에서 데이터 전송

App data : Hello

10진수 아스키 : 72, 101, 108, 108, 111

16진수 아스키 : 48, 65, 6A, 6F

2진수 : 0100 1000 0110 0101 ...

2진수 : 2^3 2^2 2^1 2^0

16진수 : 0~9 A~F

UDP checksum

목표 : 전송된 segment에서의 에러 검출(e.g. flipped bits)

Internet checksum

sender : UDP segment의 내용을 16비트 정수 시퀀스로 처리.

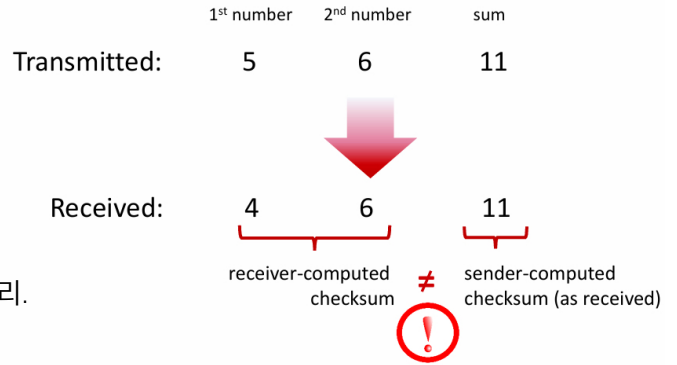
checksum : segment 내용의 추가. sum의 1의 보수.
checksum값을 UDP의 checksum field에 넣는다.

reciever : 수신한 segment의 checksum을 계산한다.

checksum의 값이 계산된 checksum과 같은지 검사한다.

-> 같지 않으면? 에러 검출

-> 같으면? 에러 없음. 그럼에도 불구하고 에러가 있을수도..?



Internet checksum: example

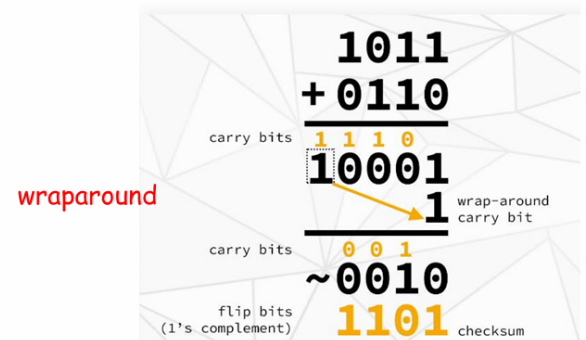
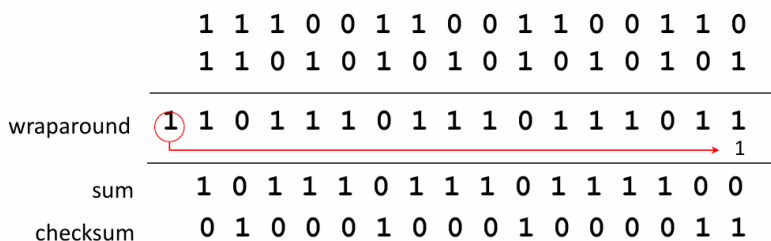
예시는 4비트 정수 2개에 대한 checksum (실제 UDP는 16bit checksum)

Carry bit이 생기면 결과에 더함

그 값을 비트 반전! 그 값이 checksum

checksum은 sum의 complement이다.

example: add two 16-bit integers



wraparound : 더했더니 자릿수(특정 범위)가 넘어가면 결괏값에 1을 더하는 방식

>>숫자를 더할 때 carryout이 일어나면 MSB(Most Significant bit; 가장 높은 자리수)를 결과에 더한다!

데이터 전송의 reliable의 원칙(이론과 구현)

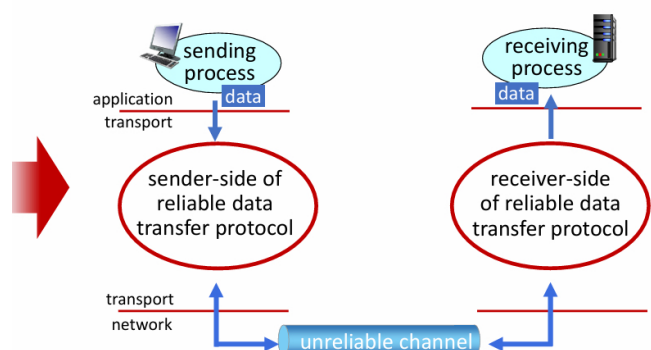


reliable service *abstraction*

unreliable한 채널을 가지고 reliable하게 만들어야 함.

reliable data 전송 프로토콜의 복잡성은 unreliable한 채널의 특성에 따라서 달라진다.

sender와 receiver는 서로가 받았는지 알 수가 없다! message로 전달되지 않는 한...



reliable service *implementation*

Reliable data transfer: getting started

sender와 reciver side의 Reliable Data Transfer protocol(rdt)의 전진적으로 개발.

단방향(unidirectional) 데이터의 전송만 고려 - 하지만 control info(제어 신호)는 양방향이다!

sender와 receiver를 특정하기 위해 FSM(finite state machine; 유한상태기계) 사용

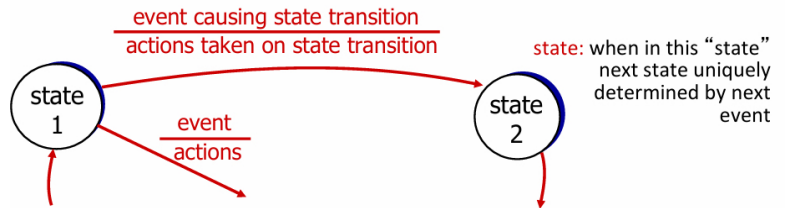
FSM : 유한상태기계; 프로그램의 순서도와 비슷!

rdt 1.0 : reliable한 채널을 통한 reliable한 전송

완전히 reliable한 채널을 가정.

bit error 없음

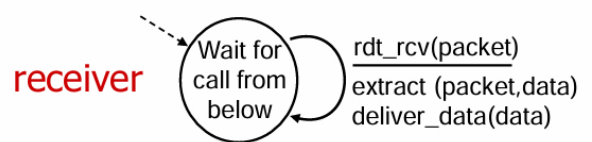
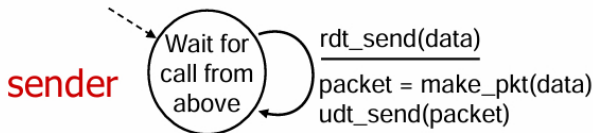
loss of packets 없음



FSM에서 송신자와 수신자를 구분.

sender는 underlying(네트워크) 채널로 데이터를 보냄

reciever는 underlying(네트워크) 채널로 데이터를 받음



rdt 2.0 : bit errors가 있는 채널

packet에서 flip bit가 있을 수 있는 채널에서의 전송

checksum으로 bit error를 탐지

에러를 복구하는 방법은?

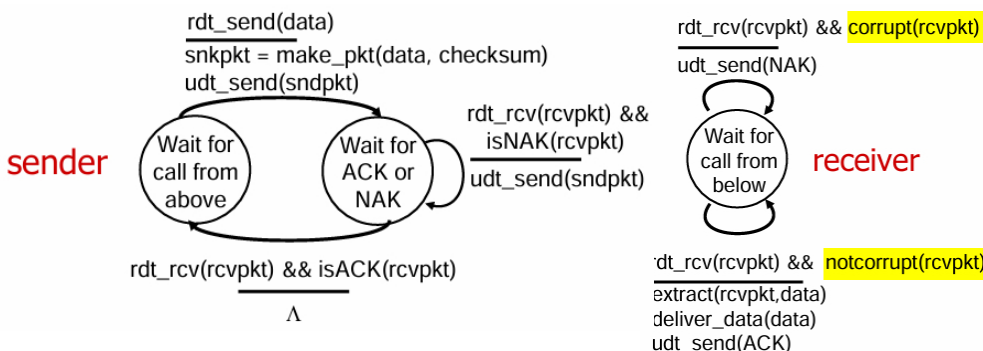
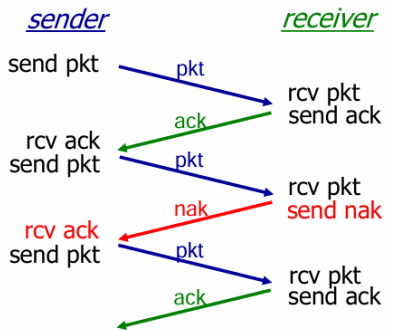
acknowledgements(ACKs) : receiver가 명시적으로 sender에게 pkt를 정상적으로 받음을 알림.

negative acknowledgements(NAKs) : receiver가 명시적으로 sender에게 pkt 수신에 에러가 있음을 알림.

sender는 NAK을 받으면 재전송(retransmit)한다.

stop and wait : sender가 패킷 하나를 보내면 receiver의 응답을 기다린다.

파이프라이닝 : 한번에 여러개 보내고 여러개 받는 것 <-> stop and wait



근데 만약 ACK/NAK이 Corrupt되면 어떡하지?

sender는 receiver에서 무슨 일이 일어나는지 알 수 없음.

duplicate(중복)될 수 있기에 재전송할 수도 없음

중복 해결 방안

sender는 ACK/NAK이 corrupt되면 재전송한다.

각각의 패킷에 **sequence number**를 추가한다.

receiver는 **duplicate(중복된) pkt**을 폐기한다.

rdt 2.1 : sequence number로 중복 제어

sender

seq number를 pkt에 추가한다.

seq number는 0과 1로 이루어져 있다. 왜 2개만으로 충분할까?

-> stop and wait 이니까 여러개 필요. 지금 보낼 건가? 다음에 보낼건가?만 판단하면 됨.

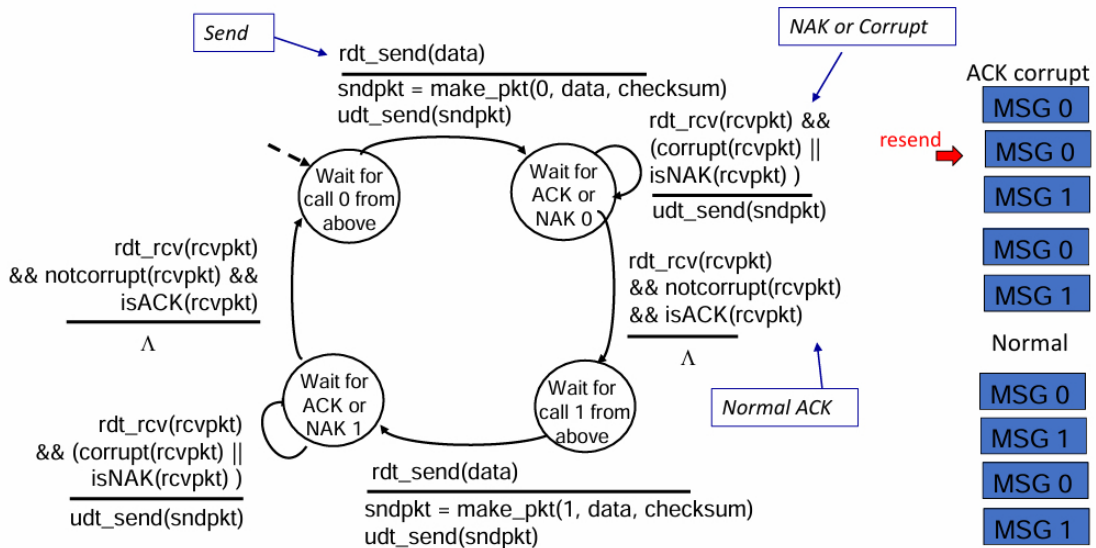
receiver

패킷이 duplicate(중복)인지 확인한다.

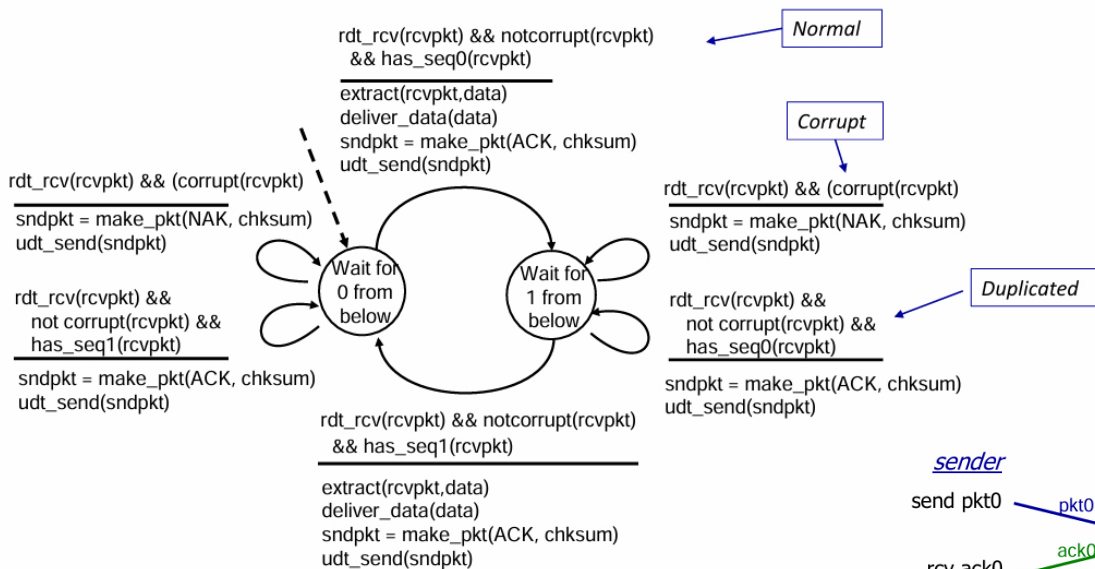
seq번호가 0또는 1로 예상되어야 하는데 맞는가?

receiver는 마지막 ACK/NAK이 sender에게 정상적으로(OK) 전달되었는지 모른다.

rdt2.1: sender, handling garbled ACK/NAKs



rdt2.1: receiver, handling garbled ACK/NAKs



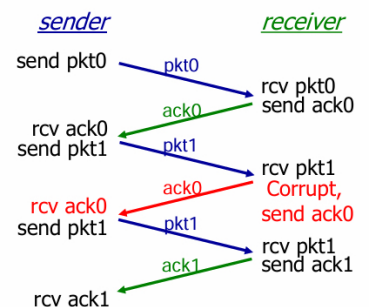
rdt 2.2 : NAK이 없는(NAK-free) protocol

기능상으로는 rdt 2.1과 같지만, ACK만 사용한다. -> 구현이 더 간단

NAK 대신, receiver는 마지막으로 수신된 OK에 대한 ACK를 보낸다!

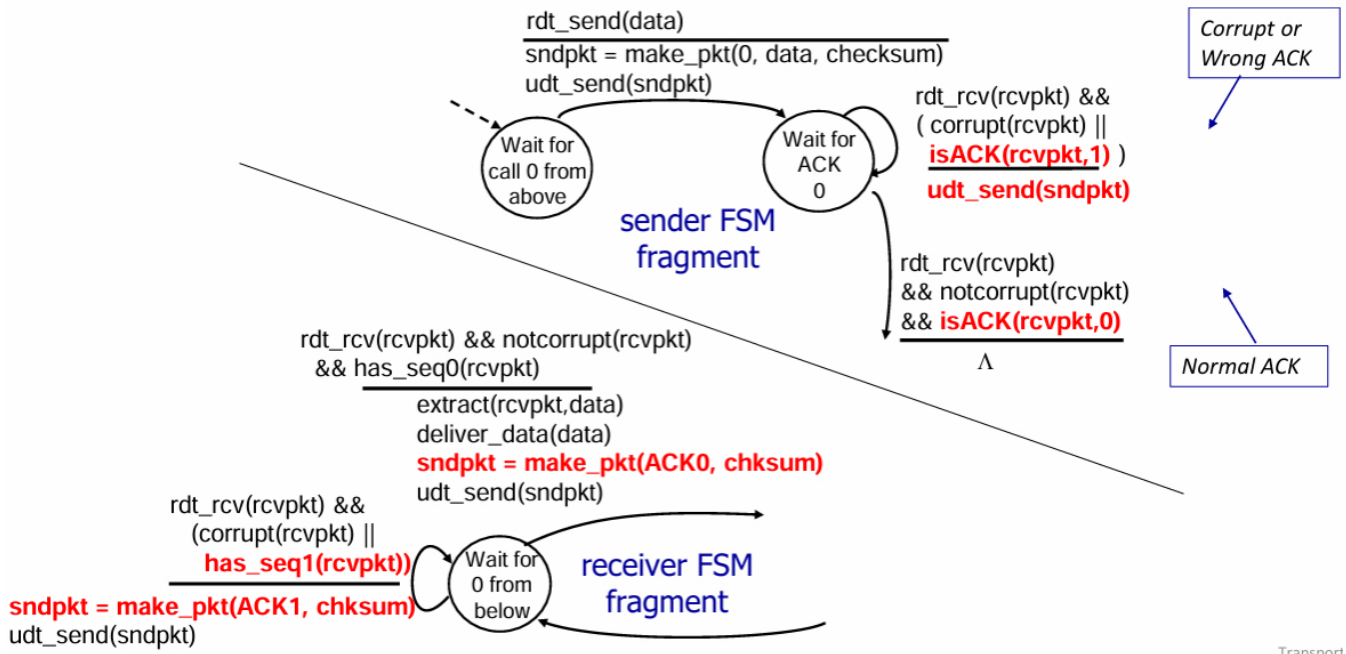
receiver는 ACK된 pkt에 seq no.를 명시적으로 포함시켜야 한다.

sender가 duplicate 중복된 ACK를 받으면, NAK과 같이 동작함(NAK으로 알고) -> 현재 pkt을 다시 전송함.



TCP는 이 방식을 사용해서 NAK을 사용하지 않는다!

rdt2.2: sender, receiver fragments



rdt 3.0 : error와 loss가 같이 있는 상황

새로운 가정 : lose packet(data나 ACK이 손실됨. 오다가 없어짐)이 있는 채널을 가정.

checksum, seq no., ACK, 재전송이 도움은 되는데... not enough!

sender가 ACK를 받기까지의 "reasonable 합리적인" 정도의 시간만큼 기다린다.

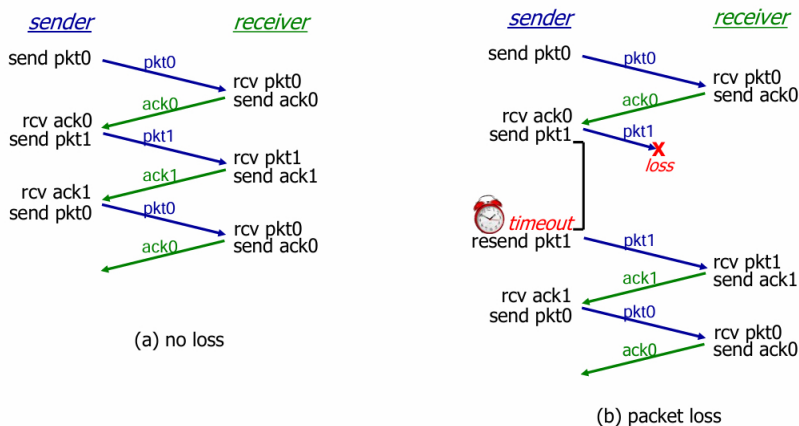
기다리고 응답이 없으면 재전송한다.

만약 pkt이나 ACK이 Lost되지 않고 delay만 됐다면...

재전송이 일어나 중복이 되지만, seq no.가 있기 때문에 문제되지 않음.

receiver는 ACK되는 패킷에 seq no.를 지정해야 한다.

타이머를 사용해 "reasonable"한 시간 이후에는 interrupt함.



Term	Symbol	Multiple	Sci Not ⁿ
Terra	T	1 000 000 000 000	$\times 10^{12}$
Giga	G	1 000 000 000	$\times 10^9$
Mega	M	1 000 000	$\times 10^6$
kilo	k	1 000	$\times 10^3$
Units	Eg. meters	1	$\times 10^0$
milli	m	1 / 1 000	$\times 10^{-3}$
micro	μ	1 / 1 000 000	$\times 10^{-6}$
nano	n	1 / 1 000 000 000	$\times 10^{-9}$
pico	p	1 / 1 000 000 000 000	$\times 10^{-12}$

U_sender : utilization 사용률 - sender가 보내는데 바쁜 시간의 비중

e.g. 1Gbps link, 15ms의 delay가 있고, 8000bit의 packet을 보낸다고 하면

$$\text{걸리는 시간(Delay)} = D_{trans} = \frac{L}{R} = \frac{8000\text{bits}}{10^9\text{bits/sec}} = 8 \times 10^3 \times 10^{-9}\text{sec} = 8 \times 10^{-6}\text{sec} = 8\text{microsecs}$$

패킷 하나가 갔다가 돌아오는 전체 시간(t)

= RTT+L/R(첫번째 패킷 비트가 도착할 때부터 마지막 패킷 비트가 도착하고 ACK 보낼때까지의 시간)

Delay에 걸리는 시간(propagation delay)을 15ms라 하면, RTT는 2배인 30ms

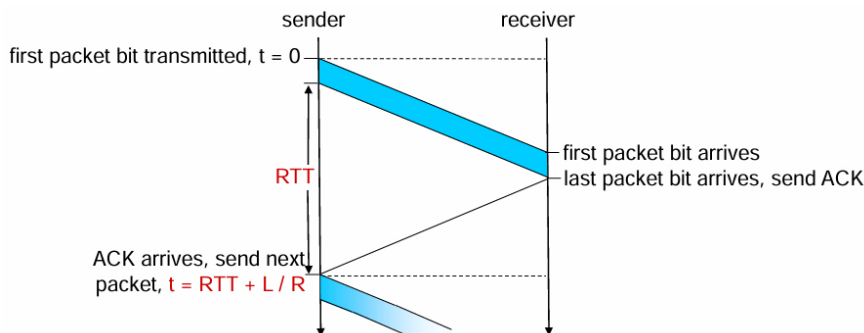
$$U_{sender} = U_{sender} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} = 0.00027 = 0.027\% \text{만 전송에 사용되고 있음}$$

>> 99.9%의 시간을 낭비하고 있음

rdt 3.0은 성능(효율)이 최악...

Protocol이 인프라(채널)의 성능을 제한함.

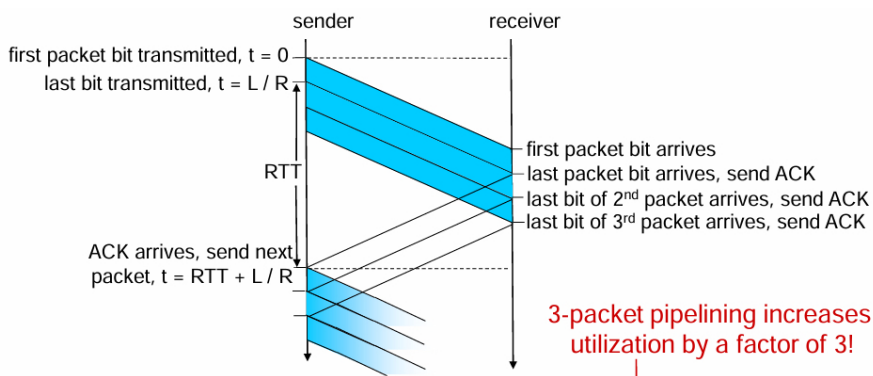
1Gbps link 속도로 260kbps밖에 못보냄(rdt)



rdt3.0: pipelined protocols operation

sender가 여러 개의 “in-flight” 진행 중인 아직 ACK되지 않은 패킷의 전송을 허용함. >> 여러개 동시에 보냄 seq no.가 늘어나야 한다.

sender와 receiver에 **buffering**이 필요하다. 오다가 순서가 뒤죽박죽 오기 때문에 다 오면 맞춰야 되기 때문



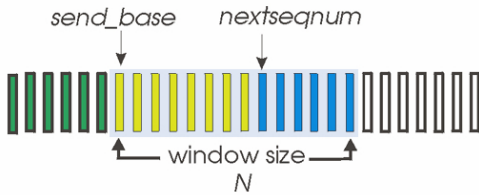
$$U_{sender} = \frac{3L/R}{RTT + L/R} = \frac{.0024}{30.008} = 0.00081$$

Pipeline에는 **Go-Back-N**과 **Selective repeat** 방식이 있다.

Go-Back-N

sender : N개의 window가 있음, 연속적으로 전송되었지만 ACKed되지 않은 패킷.

pkt 헤더의 k-bit의 seq no.가 있음.



초록 : 이미 ACK 받음

노란색 : 보냈지만 ACK 아직안옴

파란색 : 사용 가능하지만, 아직 안보냄

흰색 : 아직 안보냄

send_base : 보내기 시작한 지점

nextseqnum : 다음에 보낼 지점

window size : sender가 연속적으로 보낼 수 있는 최대 패킷의 개수. 노란색+파란색

cumulative ACK(누적 ACK) : ACK(i)는 i번째 패킷과 그 전 패킷이 모두 성공적으로 ACK되었다는 것을 의미함.

ACK(i)를 받으면 -> window가 i+1번째로 옮김

가장 먼저 간(oldest in-flight) packet에 타이머가 돌아감

만약 i번째 패킷에 timeout이 온다면, window에 i번째를 포함해 i보다 seq no.가 더 큰 모든 패킷을 재전송.

receiver :

지금까지 받은 것 중에 그 이전까지 제대로 다 받은 높은 순서의 seq no.를 보냄.

중복된 ACK를 만들고 보낼수도 있음.

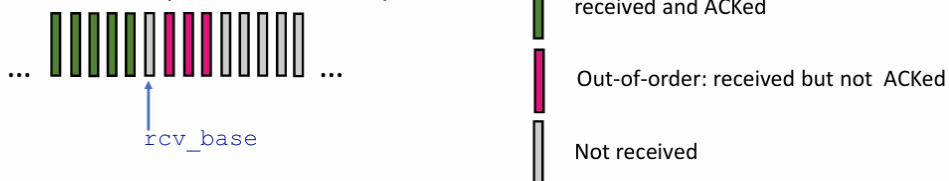
rcv_base라는 멤버 하나만 있으면 됨.(여기까지 다 잘 옴!)

out-of-order packet(순서 이상하게 온 패킷)에 대한 처리

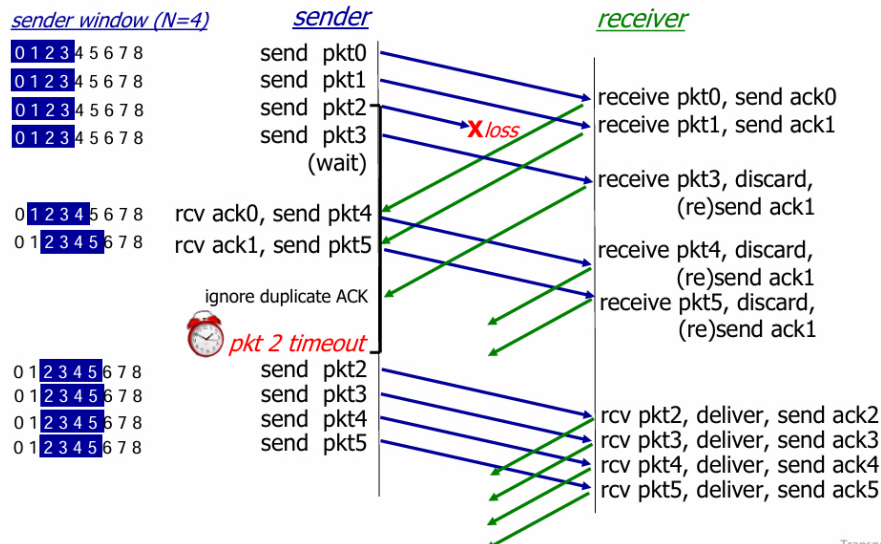
discard 버리거나, buffer 보관하거나. 구현에 따라 달라짐

순서대로 온 것 중 가장 높은 seq no.(in-order)를 계속 다시 ACK을 보냄.

Receiver view of sequence number space:



out-of-order 패킷에 대해서는 따로 처리하지 않음.



ack1(2번 보내줘)를 계속 재전송하지만 sender는 무시

pkt2가 타임아웃되면 그때 보냄

Selective repeat

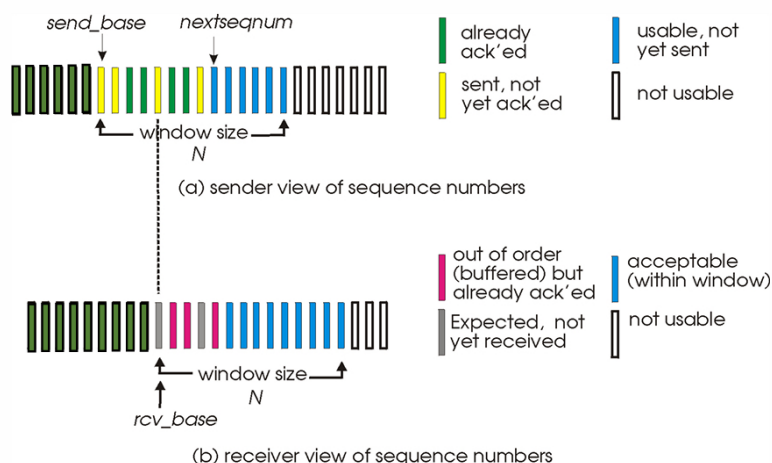
receiver

제대로 보내진 모든 패킷에 대해 ACK를 함. 정확하게 몇 번 패킷 받았는지를 알려줌.
상위 layer로 순차적으로 전달하기 위해 **packet을 buffer함!**

sender

패킷별로 타이머를 유지하고, 각각의 packet이 timeout되면 unACKed된 패킷을 재전송한다.
N개의 연속적인 seq에 대해 개념적으로 window를 유지.
in flight(진행중인) 패킷을 이 window안에 유지함.

Selective repeat: sender, receiver windows



sender

data from above:

- if next available seq # in window, send packet

timeout(i):

- Resend packet *i*, restart timer

ACK(i) in [sendbase, sendbase+N-1]:

- Mark packet *i* as received
- if the *i*-th packet is smallest unACKed, advance window base to next unACKed seq #

receiver

packet *i* in [rcvbase, rcvbase+N-1]

- send ACK(*i*)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

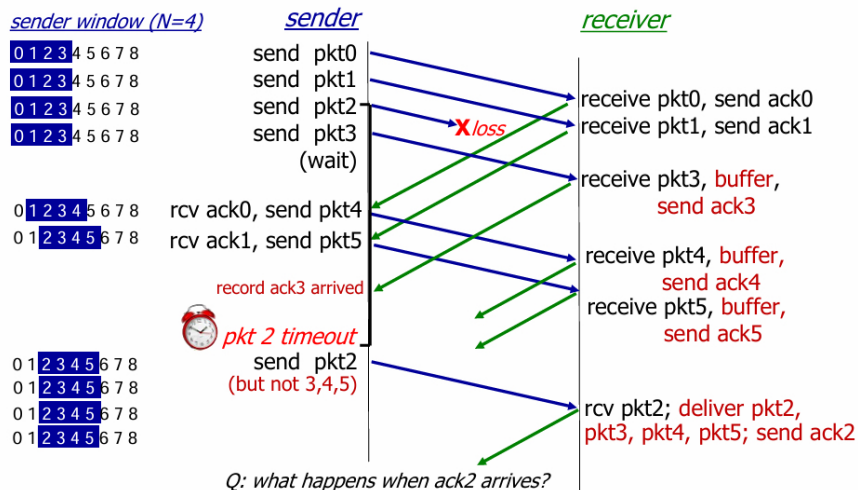
packet *i* in [rcvbase-N, rcvbase-1]

- ACK(*i*)

otherwise:

- ignore

Selective Repeat in action



9강 - 실습 SMTP

Telnet

네트워크를 통한 대화형 텍스트 기반 통신에 사용되는 네트워크 프로토콜.

TCP 23번 포트

사용자가 장치 또는 서버를 로컬에 있는 것처럼 원격으로 액세스하고 관리할 수 있음

네트워크 서비스를 CLI로 제공

대안 : SSH(Secure shell)

Wireshark

오픈소스 네트워크 프로토콜 분석기

실시간으로 네트워크를 캡처, 표시, 분석

TCP/IP, HTTP, SMTP, FTP 등등 지원

특정한 트래픽에 집중할 수 있으므로 분석의 효율성과 효과성이 높음.

실습

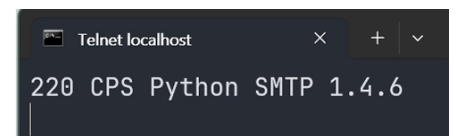
`python -m aiosmtpd -n -l localhost:1025` 1025번 포트 쓰겠다

Wireshark에서 "loopback traffic capture" 옵션 사용

터미널에서 "hostname" 쓰면 사용자 이름 나옴.

새로운 터미널에서

`telnet localhost 1025` //작성하면 텔넷 클라이언트가 작동함.



`HELO soli` //SMTP 세션을 시작하고, 서버에 host 이름을 알려줌

250 CPS

`MAIL FROM : Sungmin` //sender의 이메일 주소를 특정

250 OK

`RCPT TO : localhost` //recipient의 이메일 주소를 특정

250 OK

`DATA` //

354 END data with <CR><LF> <CR><LF>

`Subject : Test Email`

`From: local`

`To : localhost`

`this is the body of the email.`

`You can write multiple lines.`

`.`

250 OK

Wireshark에서 "tcp.port == 1025" 옵션으로 패킷을 분석

- 2xx: Successful operation
- 3xx: Further information required
- 4xx: Temporary failure, retry later
- 5xx: Permanent failure, request rejected

텔넷의 기본값은 "Character Mode"
각자의 문자는 입력되자마자 서버로 전송됨.
각각의 키 입력은 **별도의 TCP 패킷**을 생성
서버가 각각의 문자에 대해서 **ACK**를 보냄

10강 - 실습 DNS

WSL

윈도우에서 리눅스 배포판을 네이티브하게 쓸수 있게 하는 기능
윈도우에서 리눅스 명령어를 쓸수 있게 함.

DNS 분석을 하기 위해서 리눅스를 사용

nslookup

DNS를 쿼리하기 위한 커맨드라인 툴
도메인 이름이나 IP 주소를 가져오는데 사용됨

DNS가 어떻게 도메인 이름을 주는지 확인하는 데 도움이 됨.

```
nslookup naver.com           //nslookup URL
nslookup cau.ac.kr
nslookup 165.194.1.2         //nslookup IP주소
nslookup 142.250.207.110
```

```
nslookup -type=mx cau.ac.kr
```

타입 종류

- A(Address Record)
- MX(Mail Exchange)
- NS(Name Server)
- TXT(Text Record)
- CNAME(Canonical Name)

local DNS를 찾아보자

```
ipconfig /all
```

Dig

Domain Information Grouper의 약자
DNS 이름서버를 쿼리하기 위한 CLI 도구
+trace 나 **+norecurse** 옵션을 사용하여 DNS 쿼리 과정을 탐색할 수 있음.

+trace

DNS의 반복적인(iterated) process를 보여줌. 각각의 step을 단계적으로 보여줌. 루트->TLD->Authoritative...
dig +trace cau.ac.kr
dig +trace naver.com
dig @8.8.8.8 +trace cau.ac.kr

암호화된 글들 **RRSIG** by **DNSSEC**

```
.          숫자 IN NS      k.root-servers.net
Recieved 525 bytes from 8.8.8.8 in 29ms          //Root DNS서버를 8.8.8.8로부터 받았습니다.
kr.        숫자 IN NS      e.dns.kr
Recieved 618 bytes from 193.0.14.129(k.root-servers.net)  //kr DNS 서버를 k.root-servers.net으로부터 받았습니다
cau.ac.kr   숫자 IN NS      ns.cau.ac.kr
Recieved 149 bytes from 165.194.128.1(ns1.cau.ac.kr)      //cau.ac.kr을 ns1.cau.ac.kr로부터 받았습니다.
```

+norecurse

recursive 쿼리를 수행하지 않도록 함. //네임서버가 알고 있는 정보만 반환.

```
dig +norecurse cau.ac.kr
```

```
dig +norecurse naver.com
```

오류없는 답변을 얻기 위해서는?

```
dig cau.ac.kr NS          //답변에 ANSWER SECTION에 ns1.cau.ac.kr와 ns.cau.ac.kr
```

```
dig @ns.cau.ac.kr +norecurse cau.ac.kr //재귀를 비활성화해 네임서버가 알고있는것만(캐시만) 반환하도록 함.
```

11강 - 실습 DNS

실습 -- 채팅 앱

```
python server.py --port [포트번호]      파이썬 파일을 저 포트번호에다 열이라
```

```
python client.py --server [서버ip] --port [포트번호] --name [내 이름]
```

Wireshark로 TCP 패킷 캡처 - local loopback

```
tcp.port == [포트번호] && ip.src == [서버ip]
```

大

Key concept

Distributed resource : 그리드에서 컴퓨팅 파워, 저장공간, 데이터를 공유함.

Parallel processing : 큰 일을 작은 작업으로 나누어 동시에 해결

Resource sharing : 네트워크 전반에서 사용 가능한 리소스를 효율적으로 활용

Grid computing의 시나리오

A가 동영상을 재생

A의 컴퓨터에 일시적으로 동영상이 재생됨

B가 같은 동영상을 재생

B의 컴퓨터에 일시적으로 동영상이 재생됨. 이 때 A의 컴퓨터로부터 콘텐츠를 받아옴

C가 같은 동영상을 재생

C의 컴퓨터에 일시적으로 동영상이 저장됨, 이 때 A, B의 컴퓨터로부터 콘텐츠를 받아옴

실습 -- 지도

PCAP 파일을 활용해 네트워크 패킷 데이터 분석

